Toward a machine-checked proof of the claimed Sandier–Serfaty certificate: 4 of 12 stages re-executed in the Lean kernel, one reduction lemma proved, and the measured cost of the rest

(a partial formalization; the conjecture is *not* formally proved)

Xavier Callens (SocrateAI Lab)

Preprint v1, 2026-10-26 (not peer reviewed; audited by an independent AI agent, not by a person; revised after that audit)

AI-assistance disclosure. This preprint, its Lean code, its Python and C++ harness code, the LeanFlow tactic change and the claim ledger were produced by AI agents (Claude, Anthropic) under the author's direction. The author is responsible for the content. No human has audited the Lean statements, no human referee has checked the hand arguments, and every "independent" check below is made by an agent of the same AI pipeline as the producer, never by a person. In particular the same agent wrote the generators and the Lean files that this paper reports on (Section 11). The mathematics under study was itself produced by an AI system (the openai/math release) and is not peer reviewed. After the first version, a separate AI agent instance audited the paper and the Lean files (`AUDIT.md`, `STATEMENT_LOCK.json`; Section 11); that audit is also not a person's.

Abstract

The Sandier–Serfaty conjecture asks whether the triangular lattice minimizes the renormalized Coulomb energy of a planar point set against a uniform background [1]. A preprint of the openai/math release [2] (AI-generated, not peer reviewed) claims a proof from an analytic reduction plus a finite certificate in exact interval arithmetic with twelve stages. Our earlier paper [3] reproduced that certificate by running its C++ code. Here we start to *formalize* it in Lean 4 and we say exactly how far we got. **We do not prove the conjecture, and nothing in this paper says that it is proved.** Two checking levels are distinguished. **L1 (kernel re-execution):** a Lean transcription of a stage's interval computation, evaluated by the Lean kernel (`decide +kernel`, no `native_decide`), gives bit for bit the integer endpoints printed by the C++ binary. **L2 (real-number statement):** a Lean theorem that a real number lies in the interval. Counts, from the saved logs: **4 of 12 stages are complete at L1** (`init_constants`, `bcons/eps`, `calcLam`, `calcK`) [A 0002][A 0005][C 0002]; `calcLam` and `calcK` are conditional on the table enclosures and `bcons` on the lattice coefficients, all taken as premises. **1 of 12 is complete at L2** (`init_constants`: $\log 2$, π, p, R_0, a as real numbers) [A 0001]. The four L1 stages together re-execute 1,685,129 of the 1,238,829,689 interval multiplications and inversions of the C++ run, **0.136 %** [X 0001]. All 22 printed integer pairs and every assertion that the C++ makes after `calcK` (the λ range, $K < 0$, $f(\ell) > 0$, the slope, the two `global_grid` assertions, the `inside_hess` minima) plus the $|c_1| < 1$ check are kernel-checked as arithmetic on the exact endpoints (a third, weaker level, E) [A 0007]; the `core_test` bound check of stage 4 (`core.hpp`:274) is *not* covered, and the table separation check (`core.hpp`:251) is checked only window by window. One lemma of the analytic reduction, `signed_endpoints`, is proved completely, with one algebraic step and one numerical fact; it is not connected to the certificate [A 0010][A 0011]. Eleven planted false claims are rejected (must-fail files, rc 1) [X 0003]. The remaining L1 work is projected at 854–2,240 core-hours [X 0006]; the larger gap (L2 for the analytic content, and 30 further steps of the reduction) is a multi-month project [C 0007]. The conjecture is not formally proved [C 0001]. The producer was one AI agent. A separate AI agent instance (not a person) then audited the paper, with verdict ACCEPT_WITH_CORRECTIONS (`AUDIT.md`, `STATEMENT_LOCK.json`). It rebuilt 13 Lean modules from source in its own olean directory (oleans byte-identical to the producer's), re-ran the three must-fail files, re-ran the axiom aggregates and type-checked `Chain.lean`; the

28 other chunk files, `CalcK.lean` and `CalcLam.lean` were *not* rebuilt independently [C 0008]. No human has audited any statement.

Tiers and levels. Claims carry the Elenchus tiers of [3]: [A] kernel-checked Lean, axioms within {`propext`, `Classical.choice`, `Quot.sound`} (a Tier A tag attaches to the Lean statement, never to its gloss); [L] a cited source read on the cited page; [C] an argument not machine-checked; [X] a computation or observation, never supporting a mathematical claim by itself. A tag such as [A 0002] refers to `ledger/ledger.json` (prefix `P2SS`). The *checking levels* are a different axis: L1, L2 and E (endpoint arithmetic, called "A" in the project notes, renamed here to avoid a clash with the tier). A Tier A theorem can sit at level L1, L2 or E; the level says what the theorem is about. Everything resting on the openai/math manuscript is conditional on it.

1 Goal, preregistration and outcome

The conjecture and the manuscript's strategy are described in [3]: a reduction (the manuscript's Sections 2–5, 31 numbered steps audited there) to a family of scalar inequalities, which a finite certificate in exact interval arithmetic then verifies on grids. The certificate has twelve stages, from the constants $\log 2, \pi$ to the Hessian assertions on the inner domain. Proposal 2 (item A1) asked for the following, with priors fixed before any attempt (`preregistration.json`): **A1a** port the finite certificate's stages into Lean as kernel-checked inequalities with planted false stages rejected, and report the number of stages and the cost of the rest (prior 0.6 that some pilot stages succeed, 0.2 that all do); **A1b** formalize at least one complete lemma of the reduction (prior 0.35); the combination "A1 complete" had prior 0.02, and the kill criterion was that no claim of a formal proof may be made unless every stage and the whole reduction are kernel-checked.

Outcome: "some pilot stages" (prior 0.6; the preregistration does not fix a number) reached to the extent of 4 of 12 stages at L1; all stages (prior 0.2) *not* reached; one reduction lemma (prior 0.35) reached; A1 complete (prior 0.02) not reached and the kill criterion applies. Two deviations from the preregistered wording: (i) A1b was to be built "on top of `SandierSerfatyStatement.lean` and the P3 library"; `signed_endpoints` is stated for an abstract kernel G as in the manuscript and uses neither; (ii) A1a was to go "via LeanFlow"; the stages were compiled with the project script `lean_pinned.py` (pinned Lean 4.34.1, Mathlib `d13f23b7`) and LeanFlow (Section 10) was used for the twelve assertions only.

2 How interval arithmetic works in Lean here

The certificate's arithmetic is exact integer arithmetic: an interval is a pair of integers (u, v) read as $[u/Q,\ v/Q]$ with $Q = 2^{144}$ (file `IV.lean`). Sums add endpoints; products and quotients are rounded *outward*, the lower endpoint with $\lfloor \cdot/Q \rfloor$ and the upper with $\lceil \cdot/Q \rceil$, so that the true real result is never lost. Toy example with $Q = 100$: $a = [1.20, 1.30] = (120, 130)$ and $b = [0.50, 0.60] = (50, 60)$ give the four endpoint products $6000, 7200, 6500, 7800$ and hence $a \cdot b = (\lfloor 6000/100 \rfloor, \lceil 7800/100 \rceil) = (60, 78) = [0.60, 0.78]$. In Lean:

```
structure IV where  u : Int   v : Int
def fq (a : Int) : Int := a / Q                 -- floor (Q > 0, Euclidean division)
def cq (a : Int) : Int := -((-a) / Q)           -- ceiling
def IV.mul (a b : IV) : IV :=
  ⟨fq (min (min (a.u*b.u) (a.u*b.v)) (min (a.v*b.u) (a.v*b.v))),
   cq (max (max (a.u*b.u) (a.u*b.v)) (max (a.v*b.u) (a.v*b.v)))⟩
```

The C++ chooses the same two products by a nine-way sign case split; the Lean version uses min/max, and the claim that the two give the same endpoints is checked by the stage equalities below on every exercised input, not proved in general [C 0003]. Two layers are kept apart. The *computation layer* (`IV.add`, `mul`, `inv`, `rat`, `sqrt`, `hull`, ...) is executable. The *soundness layer* gives, for each operation, a theorem over $\mathbb{R}$: if reals x, y lie in the input intervals then $x \cdot y$ lies in the output interval (`mul_sound`, `add_sound`, `inv_sound`, `div_sound`, `rat_sound`, `hull_sound_left`, ...) [A 0009]. The L2 statements use the soundness layer; the L1 statements use only the computation layer.

Why `decide +kernel` and not `native_decide`. A stage theorem has the shape "`stageComputation = literal`" where the literal is an integer pair printed by the C++ (or a Boolean `chunk...= true`). `decide +kernel` asks Lean's kernel to evaluate both sides by reduction. `native_decide` would instead compile the expression and run the compiled code, trusting the evaluator and adding an axiom (on Lean 4.34.1 P1 saw it

reported as `t._native.native_decide.ax_1_1` [3]). With `decide +kernel` the `#print axioms` line of the L1 theorems reads "does not depend on any axioms" or only `[propext]` (verbatim lines in [A 0002]–[A 0005]). The kernel's native big-integer arithmetic on numerals stays in the trusted base [C 0006]. What the choice removes from the trust base of a ported stage is the C++ compiler, Boost, OpenMP and the hardware that ran the binary; it does not remove the fidelity of the transcription (Section 5).

How a long stage is encoded. Kernel memory grows with the number of operations in one theorem (we measured about 4.9 GB extra at 48 000 operations, against 0.3 GB at 3 000; [X 0006]), so a stage is cut into *chunks*: one theorem per block of loop indices, each with its own premises (the table nodes and cells it needs, written as integer literals), stated as a Boolean equality with the block's sum or hull. A *window check* in Lean ensures that the table cells used by a lookup are exactly the cells the C++ would use for the interval T computed in Lean (sorted, with the first node below T and the last above it). A *chain* theorem then states the conjunction of all chunk theorems of a stage.

3 The twelve stages and their status

Table 1 lists the C++ stages with the level reached. IV operation counts come from an instrumented single-thread copy of the C++ (counters and endpoint prints only) [X 0001]. Here the manuscript's quantities are the ones

Table 1: The twelve stages of the certificate. E: the executable assertion is evaluated on the exact C++ endpoints (arithmetic only). "Conditional L2": the real meaning of the assertion if the quantity lies in its interval.

#	C++ stage	What it certifies	IV mul+inv	Status
1	`init_constants`	$\log 2$, π; $p = \frac{1}{2}\sqrt{2/\sqrt{3}}$, $R_0 = 2p/\sqrt{3}$, $a = 2p$	2,317	**L1 and L2 complete** [A 0001][A 0002]
2	`cs`, `init_after_co`	lattice coefficients c_m, $m \le 38$; $\lvert c_1\rvert < 1$	44,268 (+ big-integer sums)	only $\lvert c_1\rvert < 1$ at E; not ported
3	`table(24000)`	jets $\tilde{f}_0, \tilde{e}$ at 6826 nodes, cell enclosures	679,762,698	not ported (premise of 6, 7, 9–12)
4	`core_test`	finite-core premise	16,826,719	not ported
5	`bcons`, `eps`	$B(\ell)$ enclosure, $\varepsilon = 10^{-11}$	609	**L1 complete** (premises c_m) [A 0003]
6	`calcLam`	λ, μ	1,440,090	**L1 complete**, conditional on table [A 0004]
7	`calcK`	$K = 2\int_0^{x_0} G(p,s)\,\mathrm{d}s$	242,113	**L1 complete**, conditional on table [A 0005]
8	asserts after `calcK`	$\lambda \in (-6.731, -6.729)$, $K < 0$, $f(\ell) > 0$, $-\pi B(\ell) - \lambda/2 > 0$	–	E; conditional L2 [A 0007][A 0008]
9	`bdsup`	derivative bounds $L_1..L_4$ on $[b, .697] \times [0, .65]$	46,277,452	printed hulls at E; not ported
10	`global_grid`, outer	$m_* - E_0 - E_I > 0$	228,976,441 (10 and 11)	final assertion at E; conditional L2; not ported
11	`global_grid`, negative part	$N_* + K + .65(\cdots) < 0$	(in 10)	final assertion at E; conditional L2; not ported
12	`inside_hess`	Hessian minima > 0 on $\mathcal{D}_0$	265,256,982	hull minima at E; not ported

described in P1 [3]; we repeat only what is needed to read each theorem. Totals: the ported stages 1, 5, 6, 7 account for 1,685,129 of the 1,238,829,689 IV multiplications and inversions of one C++ run (**0.136 %**); the denominator counts a single call of `init_constants`, and the numerator does not credit the edge-value evaluations of `init_after_co` that stage 5 also re-executes, so the share is conservative [X 0001]. Interval work is not the same as proof content: the unported stages are the ones that carry the analytic burden.

4 Step-by-step: what each ported stage certifies

4.1 Stage 1: `init_constants` (levels L1 and L2) [A 0001][A 0002]

What it certifies. Enclosures for five constants used by everything else: $\log 2$, π, $p = \frac{1}{2}\sqrt{2/\sqrt{3}}$ (so $p^4 = \frac{1}{12}$), $R_0 = 2p/\sqrt{3}$ ($R_0^4 = \frac{4}{27}$) and $a = 2p$ ($a^4 = \frac{4}{3}$). *How the C++ does it.* $\log 2 = 2\,\mathrm{atanh}(1/3) = 2\sum_{i\ge 0} \frac{(1/3)^{2i+1}}{2i+1}$, summed for 200 terms in interval arithmetic and widened by $[-1,1]/Q$ for the tail; $\pi = 16\arctan\frac{1}{5} - 4\arctan\frac{1}{239}$ (Machin), 150 terms of each alternating series, widened by $[-2,2]/Q$ for each tail; the roots by an integer square root of the scaled endpoints. *L1.* The theorems `ln2_comp_eq`, `pi_comp_eq`, `p_comp_eq`, `R_comp_eq`, `a_comp_eq` say that the Lean transcription of these loops, evaluated by the kernel, equals the endpoints printed by the C++ dump driver (a program compiled against the unmodified corpus headers, [X 0001]). *L2.* The theorems `log_two_mem`, `pi_mem`, `p_mem`, `R_mem`, `a_mem` state that the *real numbers* $\log 2, \pi, p, R_0, a$ lie in the intervals. They are proved from Mathlib (the bound on the remainder of the logarithm series, Machin's formula together with the arctangent series and a geometric tail bound, and the fourth-power identities above) plus kernel-decided rational comparisons, *independently of the C++ algorithm*: the L2 proof checks the C++ output, not its program. This is the only stage whose output is connected to the real numbers it names. A bridge file shows that the certificate's constants contain the scale triScale of the P1 statement file [A 0012].

4.2 Stage 5: `bcons` and `eps` (level L1) [A 0003]

What it certifies. The enclosure `bcons` of the edge value $B(\ell)$ used in the sign assertion $-\pi B(\ell) - \lambda/2 > 0$, and the truncation allowance $\varepsilon = 10^{-11}$. The C++ forms $-\log R + \frac{\pi}{2}R^2 + \sum_{m\le 38} VR_m + \varepsilon$ (with R the C++ variable of that stage; see `Bcons.lean`), where VR_m are the 38 edge values computed in `init_after_co` from the lattice coefficients c_m. *In Lean.* `bcons_comp_eq` re-executes the interval log (range reduction plus the 200-term series), `pow`, the binomial factors, the Horner evaluations, the 38 values VR_m and the final sum, and finds the C++ endpoints. The c_m (stage 2) are premises: they enter as integer literals.

4.3 Stage 6: `calcLam` (level L1, conditional on the table) [A 0004]

What it certifies. The constants λ and μ of the calibration, from the table functions $\tilde f_0$ and $\tilde e$ (stage 3) by two midpoint sums with $s_0 = .01$: $B = \frac{p}{R/2}\big(f(R) - f(p)\big) - \frac{R/2}{p}\,e(R)$ (up to the origin correction and the midpoint-rule allowances), $\lambda = B/(pR/2)$, $\mu = f(R) - e(R)/3 - \lambda R^2/2$. *In Lean.* Each of the $2\times 45\,000$ midpoint steps is a table lookup: the cells meeting a query interval, linear interpolation between the two nodes plus the remainder allowance $H^2/8\cdot|\tilde f_0''|$ and $+\varepsilon$, hulled over the cells. They are grouped in 300 chunks of 150 steps for each of the two sums (`chunkFacc`, `chunkEacc`). `calcLam_chunk_sums` forms the exact sums of the chunk values; `calcLam_eq` states that the scalar remainder of the stage (the hull of $\tilde f_0''$ over the cells meeting $[p, \sqrt{p^2+s_0^2}]$, the derivative bounds over the 1997 cells meeting $[p, R]$, both allowances, then λ, μ) gives exactly the C++ endpoints. `calcLam_stage` is the conjunction of the 600 chunk theorems and `calcLam_eq`.

4.4 Stage 7: `calcK` (level L1, conditional on the table) [A 0005]

What it certifies. $K = 2\int_0^{x_0} G(p,s)\,\mathrm{d}s$ with $x_0 = R/2$ and $G(d,s) = \frac{d}{t^2}\big(f_0(t) - \frac{\lambda}{2}t^2 - \mu\big) - \frac{s^2}{dt^2}\,e(t)$, $t = \sqrt{d^2+s^2}$. The C++ uses the midpoint rule on 3103 cells of width $h = (R/2)/3103$ and bounds its error by the classical $h^2(R/2)\sup|G_{ss}|/24$, with the supremum enclosed by the hull of G_{ss} over 1552 intervals of width $(R/2)/1552$ (a second function, $G(\cdot,\cdot,\mathrm{true})$, returning $G, G_d, G_s, G_{dd}, G_{ss}$). *In Lean.* `calcK_mesh` states the two sample counts; 97 chunk theorems `chunkS` (32 midpoint terms each, the last 31) give the exact chunk sums of $h\,G(p,(i+\frac{1}{2})h)$; 194 theorems `chunkF` (8 hull terms each) give the hull of G_{ss}; `calcK_K_eq` states that $K = 2(k_c + [-e_r, e_r])$ with $e_r = h^2(R/2)|\mathrm{ss}|/24$ equals the C++ K. Each lookup re-checks in Lean that its window of cells is the C++ range.

```
theorem calcK_K_eq : Kfinal = KCpp := by decide +kernel
set_option maxRecDepth 100000 in
/-- kc chunk: indices 0..31, table cells 3003..3003. -/
theorem chunk_S0 : chunkS S0_nodes S0_cells 0 32 S0_val = true := by
  decide +kernel
```

4.5 Stages 8, 10, 11, 12 and the 22 printed pairs (level E and conditional L2) [A 0007][A 0008][A 0016]

What is checked. The exact 144-bit endpoints produced by the C++ for $\lambda, \mu, K, B(\ell), f_0(\ell)$, the slope, $f(\ell)$, the five hulls of `bdsup`, `gmin`, `drneg`, `factor`, `er_drop`, the left-hand sides of the two grid assertions and the Hessian minima are written into `Asserts.lean` as integer literals. `printed_pairs` proves that each of the 22 pairs saved in the record is $(\lfloor 10^7 u/Q \rfloor, \lceil 10^7 v/Q \rceil)$ of the exact interval, and `cpp_asserts` evaluates, on those endpoints, the assertions that the C++ makes after `calcK` (`grids.hpp`:347–348), the final `global_grid` assertion (336), the `inside_hess` hull minima (equivalent to its per-cell `failures==0` assertions, because the hull lower endpoint is the minimum over the cells) and the $|c_1| < 1$ check (`core.hpp`:293):$\lambda \in (-6.731, -6.729)$, $K < 0$, $f(\ell) > 0$, slope > 0, $(\text{gmin} - \text{factor}).u > 0$, $(\text{drneg} + \text{er_drop} + K).v < 0$, $g_{dd}.u$, $g_{xx}.u$, $\det .u > 0$ and $|c_1| < 1$. `grid_margins` adds the quantitative margins ($\geq 1.20 \times 10^{-5}$ and $\leq -4.585 \times 10^{-4}$). **Not covered:** the `core_test` bound check of stage 4 (`core.hpp`:274, `bc.v<6000Q && dsum.v<2e6Q`, the finite-core premise), whose operands were never exported; and the table separation check (`core.hpp`:251), which is checked only window by window through `sortedOK`. *What this is not.* Arithmetic about integers. It does not show that the C++ *computed* these integers from the manuscript's formulas (stages 3, 4, 9–12 are not ported). `lambda_real`, `K_real`, `fl_real`, `slope_real`, `gmf_real`, `neglhs_real` state the real meaning of each assertion *if* the quantity lies in its interval; whether it does is the job of the stage that produced the interval. The twelve assertions were also given to the LeanFlow tool as a data certificate (Section 10).

5 The chain lemma, and what it does not check

`premises_chain` [A 0006] is a kernel-checked equality of the premises that the stage files carry as literals: the p, R written into `calcK` equal the Constants outputs; the λ, μ written into `calcK` equal the `calcLam` outputs; the K of `Asserts.lean` equals the `calcK` output; `bconsC` equals the `bcons` output; and $\varepsilon = [-\text{rat}(10^{-11}).v, \text{rat}(10^{-11}).v]$ in both stages. `calcK_stage` and `calcLam_stage` each bundle all chunk theorems of a stage. Together, stages 1, 5, 6, 7 compose. After the audit, a new file `ChainLink.lean` (the 52 locked files are unchanged) adds two theorems that tie the stage-8 literals to the L1 outputs: `slope_eq` (the slope literal equals $-\pi B(\ell) - \lambda/2$ recomputed from the `Constants`, `bcons` and `calcLam` outputs) and `fl_eq` ($f(\ell) = f_0(\ell) - \lambda\ell^2/2 - \mu$, still conditional on $f_0(\ell)$, a table value); both depend on no axioms [A 0017]. The audit found that the statements could be linked this way; the producer then added them.

What is *not* checked [C 0004]:

- **Chunk coverage.** That the chunk index ranges tile the full index sets (97 chunks of 32 for 3103 midpoint indices, 194 of 8 for 1552, 300 + 300 of 150 for 45 000) is guaranteed by the generator scripts, not by Lean. The ranges are visible in the statements, and a script found them contiguous with no gap or overlap [X 0007]; a reader can redo this from the statement of `calcK_stage` and `calcLam_stage`.
- **Inputs taken as given.** The table enclosures (stage 3: nodes and cells, as literals), the lattice coefficients c_m (stage 2) that enter stage 5, the edge position ℓ, and all outputs of the unported stages. So stages 6 and 7 are *conditional on the table* and stage 5 on c_m. The table enters as 891 separate literal windows (18,455 node and cell literals) plus scalar copies; Lean does not check that these are consistent copies of one table. The auditor's script compared every node and cell literal with the C++ dump (each window a contiguous index run, its cells exactly those between its nodes) and found no mismatch (an X observation, [X 0010]). Stage-output bit equality alone would not detect every error: a 1-ulp shift of all node f lower endpoints is absorbed by the outward rounding of hG and is accepted by the chunk theorem, while the direct literal comparison detects it. The `core_test` bound check (`core.hpp`:274, stage 4) is a runtime check of the C++ whose operands were never exported, so no Lean theorem evaluates it.
- **Real meaning** of any interval except those of stage 1.
- **Transcription fidelity** (C++ to Lean) beyond bit equality on the exercised inputs [C 0003].

6 One lemma of the analytic reduction: `signed_endpoints` [A 0010]

The manuscript's Lemma `dual:signed-endpoints` (Section 3, lines 567–611 of `03-geometry.tex`) turns two scalar inputs on a kernel $G(d, s)$ into a bound for signed pairs of angles. It is the step that lets the certificate's `global_grid` assertions (the *outer* bound $2g_d \geq K$ and the *negative-part* bound) control the sector energies.

Setting: $G(d,\cdot)$ is even and locally integrable for $d \geq d_0$, and $g_d(x) = \int_0^x G(d,s)\,\mathrm{d}s$. `gPrim G d x` is $\int_0^x G(d,s)\,\mathrm{d}s$

Table 2: Hypotheses and conclusion: the manuscript's wording beside the Lean statement (verbatim from `SignedEndpoints.lean`).

Manuscript	Lean
$K < 0$, $d_0 > 0$	`theorem signed_endpoints (G : ℝ → ℝ → ℝ) (K d₀ : ℝ) (hK : K < 0) (hd₀ : 0 < d₀)`
$G(d,\cdot)$ even for $d \geq d_0$	`(heven : ∀ d, d₀ ≤ d → ∀ s, G d (-s) = G d s)`
locally integrable	`(hint : ∀ d, d₀ ≤ d → ∀ a b : ℝ, IntervalIntegrable (G d) volume a b)`
$2g_d(x) \geq K$ for $x \geq 0$ (outer input)	`(hpos : ∀ d, d₀ ≤ d → ∀ x, 0 ≤ x → K ≤ 2 * gPrim G d x)`
$\int_0^\infty \max\{0,-G(d,s)\}\,\mathrm{d}s \leq -K$ (negative-part input)	`(hneg : ∀ d, d₀ ≤ d → ∫⁻ s in Ioi 0, ENNReal.ofReal (-G d s) ≤ ENNReal.ofReal (-K)) :`
conclusion for $\lvert\alpha\rvert, \lvert\beta\rvert < \pi/2$, $\alpha+\beta > 0$	`∀ d, d₀ ≤ d → ∀ α β : ℝ, |α| < π / 2 → |β| < π / 2 → 0 < α + β → K ≤ gPrim G d (d * Real.tan α) + gPrim G d (d * Real.tan β)`

(the manuscript's g_d). Two choices deserve a remark. The negative-part hypothesis is a `lintegral` of `ofReal(-G)` (which equals $\max\{0,-G\}$ pointwise), valued in $[0,\infty]$: no integrability side condition is needed, and an infinite integral *violates* the hypothesis instead of producing the Bochner integral's junk value 0. "Locally integrable on $\mathbb{R}$" is stated as interval integrability on every bounded interval, which is equivalent.

The proof, step by step (as in the manuscript, and as in the Lean file). (1) *Oddness*: from evenness of $G(d,\cdot)$ and the change of variable $s \mapsto -s$, $g_d(-x) = -g_d(x)$ (`gPrim_neg`). (2) *Both angles nonnegative.* Then $x = d\tan\alpha \geq 0$ and $y = d\tan\beta \geq 0$, and the outer input gives $g_d(x) \geq K/2$ and $g_d(y) \geq K/2$; add. (3) *Mixed signs.* If $\alpha < 0$ then $\alpha+\beta > 0$ forces $\beta > -\alpha > 0$; put $u = -d\tan\alpha > 0$ and $v = d\tan\beta$. Since tan is strictly increasing, $\beta > -\alpha$ gives $u < v$. By oddness $g_d(d\tan\alpha) + g_d(d\tan\beta) = g_d(v) - g_d(u) = \int_u^v G(d,s)\,\mathrm{d}s$. (4) *Core estimate* (`integral_ge_of_negPart`): $\int_u^v G \geq -\int_u^v G^- \geq -\int_0^\infty G^- \geq K$, using the negative-part input. (5) The case $\beta < 0$ is symmetric (exchange the roles of α,β); both negative is excluded by $\alpha+\beta > 0$. Without the hypothesis $\alpha+\beta > 0$ the claim is false: for $G \equiv 1$, $K = -1$, $d = 1$, $\alpha = \beta = -\pi/4$ the left side is $-2 < -1$ (`ctl_signed_without_sum_false`, a kernel-checked refutation), while the same $G \equiv 1$ satisfies all other hypotheses, so the theorem is not vacuous (`ctl_signed_hyps`) [A 0014]. Fidelity to the manuscript was compared item by item by an agent, and no human has compared the Lean statement with the TeX [C 0005]. Two smaller results in the same file [A 0011]: `assembly_algebra` (the final algebraic step of the manuscript's Proposition `dual:assembly`: sector bounds $J(S) \geq K + \lambda A(S) + \mu\theta(S)$ over a finite family with $\sum A = N$, $\sum\theta = 2\pi N$, at most $6N$ members and $K \leq 0$ give $\sum J \geq N(6K + \lambda + 2\pi\mu)$, the geometric inputs being hypotheses) and `b_lt_dstar` ($0.28209 < 1/(2\sqrt{\pi})$). **What is missing:** the lemma is stated for an abstract G; the concrete G requires the radial data f_0, e of the manuscript's Section 4, which are not defined in Lean, and nothing here derives the two hypotheses from the certificate.

7 Controls: planted false stages must fail

Every claim of Sections 4–6 was paired with a planted false variant. Two kinds of evidence are kept apart. A *refutation* is a kernel-checked proof that the planted claim is false ([A 0013]); a *rejection* is the observation that a file asserting the planted claim fails to compile, with rc 1 ([X 0003]). The three must-fail files fail only with the

errors listed, each mapped to one planted theorem (no other error type appears): ControlsReject.lean 7 errors, CalcKReject.lean 2, CalcControlsReject.lean 2. *The two ulp refutations compile in `CalcKReject.lean` (the file

Table 3: Planted false stages. "refuted" = honest `ctl_*` theorem proved; "rejected" = the `rej_*` file fails (rc 1).

Planted claim	Level	Refuted by	Rejected as
$\log 2$ with 199 terms instead of 200	L1	`ctl_ln2_199_ne`	`rej_ln2_199`
π without the $[-2,2]/Q$ slack	L1	`ctl_pi_noslack_ne`	(refutation only)
π lower endpoint $+2^{100}$ ulps lies below Machin's value	L2	`ctl_pi_shift_false`	`rej_pi_shift` (decide failed to reduce; falsity by the control)
printed pair $(121, 162)$ for gmin−factor	E	`ctl_pair_false`	`rej_pair`
gmin−factor $\geq 1.21 \times 10^{-5}$	E	`ctl_gmf_121_false`	`rej_gmf_121`
$(\mathrm{drneg} + 2\,\mathrm{er_drop} + K).v < 0$	E	`ctl_erdrop_x2_false`	`rej_erdrop_x2`
det ≥ 0.0087	E	`ctl_det_false`	`rej_det`
lookup remainder $H^2/8 \to H^2/4$ (calcK pilot)	L1	`ctl_G_eight_to_four`	`rej_G_eight_to_four`
calcK chunk value ±1 ulp (S0, F0)	L1	`ctl_S0_ulp_ne`, `ctl_F0_ulp_ne`*	`rej_S0_ulp`, `rej_F0_ulp`
calcK stage, $/24 \to /12$ in e_r	L1 stage	`ctl_calcK_er12_ne`	`rej_calcK_er12`
calcLam stage, $/24 \to /12$ in the $\tilde{f}_0''$ allowance	L1 stage	`ctl_calcLam_evipp12_ne`	`rej_calcLam_evipp12`
`signed_endpoints` without $\alpha + \beta > 0$	A1b	`ctl_signed_without_sum_false`	(refutation only)
shifted-π claim by `sorry` / by a planted axiom	axiom gate	–	`#print axioms` shows `sorryAx`, respectively `ctl_cheat`: outside the whitelist (observation of the axiom gate, by design) [X 0009]

has exactly the two `rej_*` errors), but that file prints no axiom report. Ten of the eleven rejections are "`decide` proved that the proposition is false"; `rej_pi_shift` is the exception noted above. The two *stage-level* controls (bold) perturb one constant inside a whole 97-chunk or 600-chunk stage and show that the stage statements are not insensitive to the allowance they certify. The Hessian and `core_test` stages have no L1 control because they are not ported.

8 What the remaining work costs

Measured on the pinned toolchain, on two cores, at a machine load average of 12–16 from other sessions, so every figure is a range [X 0006]. Per-unit costs from the compile logs: a simple $G(d, s)$ with lookup 0.17–0.24 s (the four S files hold 3103 evaluations; per-file range after subtracting the 9.3 s import-only baseline of `BenchIV0.lean`; average 0.21 s), a value lookup in `calcLam` 0.11 s (90 000 steps in the twenty files), $G(d, s, \mathrm{true})$ 0.20–0.50 s (the eight F files hold 1552 evaluations; same baseline subtraction; average 0.32 s). These ranges come from the compile logs only; the earlier trial-chunk figures of the source notes are not used because they cannot be recomputed from a saved log. Throughput of IV multiplications and inversions: lower bound 148 ops/s (the 1,682,203 operations of `calcK` and `calcLam` over the process time of all their files: 1,281.8 s and 10,080.6 s, including the core and final files; the chunk files alone are 1,251.3 s and 9,910.1 s), upper bound 612 ops/s (kernel benchmark files `BenchIV*.lean` with 3k–48k operations; the three files give 612, 506, 360 ops/s after subtracting the import baseline). One theorem needs about 0.3 GB extra at 3k operations and 4.9 GB at 48k (3.3 GB import baseline), so we keep theorems at or below about 10^4 operations. That is about 18–47 days on two cores, or 0.6–1.5 days on 64 cores, for L1 of the unported computations (stage rows 2–4 and 9–12 of Table 1; row 8 is assertion-only and rows 10 and 11 share one computation, so the table has six lines, not eight). **The lower end of the range is not conservative, and 854 core-hours is not a lower bound.** The independent audit rebuilt files on a quiet machine (load average about 1.3) and measured compile times 2.2–2.7 times shorter than

Table 4: Projected cost of L1 for the unported stages (counts of the instrumented run × measured rates).

Stage	Work	Basis	Core-hours
`global_grid`	7,895,832 G	per-G cost	381–534
`inside_hess`	2,451,479 $G(\cdot,\cdot,\text{true})$	per-G_{full} cost	133–338
`bdsup`	421,947 $G(\cdot,\cdot,\text{true})$	per-G_{full} cost	23–58
`table`	6.80×10^8 mul+inv (and 5.2×10^8 additions, 42,014 logs)	rate, serial; needs per-node state witnesses	309–1275
`core_test`	1.68×10^7 mul+inv	rate	8–32
`cs`	136,594 lattice points	estimate: mode-0 arithmetic measured at 6.8 ms/point	1–3
Total			**854–2,240**

the producer's logs (CalcKS0 55.0 s against 148.2 s, CalcKF0 21.7 s against 48.5 s, CalcLamE3 242.5 s against 630.5 s, Bcons 130.5 s against 287.1 s). The simple-G cost in CalcKS0 comes to about $(55.0-9.3)/800\approx0.057$ s, below the 0.17–0.24 s used for `global_grid`; at that rate the `global_grid` line alone falls to about 125 core-hours. The "lower bound" of 148 ops/s holds only for the load under which it was measured. The load caveat applies in both directions: the lower rates are pessimistic on a quiet machine, and the benchmark rates are optimistic for lookup-heavy code. Read the projection as an order-of-magnitude estimate (hundreds to a few thousand core-hours), not as a bracket. The `table` line also hides a design problem: the stage is a serial propagation of an ODE-like recurrence over 6826 nodes, and a kernel check needs the intermediate state of every node supplied as a witness (about 41 intervals per node).

What L1 would still not give. Even with all stages at L1, the result is a statement about integers that a C++ program printed. The larger gap is **L2**: Lean definitions of f_0 and e (the manuscript's Section 4 radial data) and of the lattice coefficients c_m as lattice sums with the Section 5 tail bounds, and soundness proofs for the jet arithmetic, the ODE propagation of `table`, the midpoint and interpolation remainders (the pinned Mathlib has a trapezoidal-rule file; a string search finds no midpoint-rule remainder, [X 0008]), the lookup, and the grid arguments of the manuscript's Theorem `comp:scalar`. After that, the bridge from `global_grid`'s assertions to the two hypotheses of `signed_endpoints`, and the reduction itself.

9 What remains of the reduction

Of the 31 numbered steps audited in P1 (R1–R10, G1–G7, K1–K8, T1–T6), only G6 is complete in Lean, and the algebraic part of G7; the other 29 steps and the geometric part of G7 are untouched, i.e. 30 items remain, plus the numerical fact $b<d_*$ which belongs to the manuscript's Section 7 and is outside the 31 [C 0007]. Our estimate of what Mathlib lacks, from reading the pinned version and a string search [X 0008]:

- **R1–R10 (square tori to the plane).** Weak div / curl equations with point charges and their regularity; the cited results of [1] (Theorem 1, Lemma 4.7, Proposition 4.9) are paper-level theorems with no Mathlib counterpart. Multi-month by itself.
- **G1–G5, G7 (torus minimizers, Voronoi sectors, dual identity).** A torus Green function (absent), a strong minimum principle for $W^{2,2}$ supersolutions, Voronoi tessellations of periodic point sets and Euler's formula on the torus (the scan finds one docstring mention of Voronoi diagrams and no development), Green's identities on punctured domains.
- **K1–K8 (calibration).** Mathlib has a Weierstrass $\wp$-function file, a starting point for the Green function expansion; the explicit radial data and the primitive/stationarity identities are elementary but long. Weeks to months.
- **T1–T6 (tails).** Elementary real analysis (geometric and ratio tails, Leibniz bounds); P1 checked every constant in exact rationals; formalizing is long but routine (weeks).

These are estimates [C 0007]. Honest summary: months, not days, and the certificate-side L2 work is comparable to the reduction itself.

10 LeanFlow: the `decide_kernel` tactic

LeanFlow is our Lean tooling project. In P1 we added a fail-closed data-certificate generator that turns printed interval data into Lean-checked integer arithmetic; [3] also reports a fail-open hole in the checker for arbitrary sources and its fix. The generator's `decide` is evaluated by the elaborator; the numbers of this paper are 2^{144}-scale integer literals, for which kernel evaluation is much faster (the project documentation's statement). We therefore added to the generator the tactic name `decide_kernel`, rendered as `decide +kernel`, with the same fail-closed discipline: any other tactic string (`native_decide`, `decide +native`, text containing `;`) is an `invalid-claim` and never reaches Lean, the axiom audit still applies to every theorem, and a false statement gives the same "`decide` proved the proposition false" error and is classified `false-statement` as before. The change is LeanFlow commit `06084ef5c40057c66c476feba18ea3da8fb150ba` (documentation, source and tests: 4 files). The test suite, rerun read-only at the paper stage, passed 94 tests in 23.84 s, including a positive case and a planted false case at the 2^{144} scale [X 0005].

Applied to the twelve assertions on the exact endpoints, the run returns status `ok`, code `verified`, tier A, every theorem "does not depend on any axioms" (1.08 s) [A 0016]. The planted copy ($\mathrm{gmin} - \mathrm{factor} \geq 1.21 \times 10^{-5}$) returns `rejected`, `false-statement`, failing exactly `cert_5` (0.82 s) [X 0004]. As in P1, the tier of such a certificate covers the arithmetic about the stated integers only; it does not cover where they came from, and the checker for arbitrary sources is not a sandbox.

11 Limitations

- **Independent AI audit; no human audit.** One AI agent wrote the generators, the Lean files and the paper and ran the original checks. A separate AI agent instance (not a person) then audited the work (`AUDIT.md`, verdict ACCEPT_WITH_CORRECTIONS; hashes of all 52 Lean files and verbatim text of four headline statements in `STATEMENT_LOCK.json`). It rebuilt 13 modules from source in a separate olean directory (among them IV, Constants, Asserts, Bcons, SignedEndpoints, Controls, CalcKCore, CalcKS0, CalcKF0, CalcLamCore and CalcLamE3) with oleans byte-identical to the producer's; re-ran the three must-fail files (7, 2 and 2 errors, rc 1); re-ran the axiom aggregates (615, 294 and 44 reports, none outside the whitelist); and type-checked `Chain.lean` against the prebuilt oleans. **Not rebuilt independently:** the other 28 chunk files, `CalcK.lean` and `CalcLam.lean` (about 3 h); for those the evidence is the producer's logs plus the determinism shown by the byte-identical oleans that were rebuilt. The audit did not re-run LeanFlow. No human has audited any statement, and the 16 `LEDGER_UNAUDITED_TIER_A` flags stay open, because an agent audit is not the person's certification that flag asks for [C 0008].
- **Instrumented C++ copy.** The endpoints of gmin, drneg, factor, er_drop and the Hessian hulls (the data of stages 10–12 at level E) and all operation counts come from a copy of the C++ with counters and endpoint prints added (its log reproduces all 22 record pairs); the remaining endpoints come from a dump driver over the unmodified headers. The Lean theorems are only as good as these printed integers.
- **Stale object file.** `CalcKCore.lean` was rewritten byte-identically by its generator after its `.olean` was built (sha256 equal to a fresh regeneration), so the file that was compiled is the file on disk.
- **Conditional stages.** `calcLam` and `calcK` are conditional on the table enclosures, `bcons` on the lattice coefficients; the chunk tiling is not checked in Lean.
- **Load.** Timings were taken at load average 12–16; the cost table is a range, not a benchmark.
- **Statement adequacy.** No human has certified that any Tier A statement means what its gloss says; the gate reports this as one flag per Tier A row (Section 13). The fidelity of `signed_endpoints` to the TeX is an agent's comparison.
- **Scope of the manuscript.** All of the above treats the manuscript as given. We have not found, and have not looked for, an error in it; absence of a finding is not a finding.
- **Literature.** We did not search for work on the conjecture after the cited papers. The excerpt of [1] is a transcription from P1, not re-read here.

12 Reproducibility

The Lean sources and the A1 results are committed in AutoevolveAI at `d4cc150a7c2e34058c24e92d08c2485b4d683f08` (branch `worktree-openai-math-discovery`; `git status` shows both paths clean and all

52 Lean files tracked); only papers/p2_sandier_serfaty_lean/ and the new file ChainLink.lean (sha256 ecb144e19dfde7001fa608e8e2f62e6e43a6a6a169ab5bfa946346add4a1d136; added after the audit, compiled rc 0 with no axioms, data/compile_ChainLink.json) are uncommitted. Lean sources: formal_cert/Programmes/SandierSerfaty/ (52 files; sha256 of every file in data/result.json; IV.lean 1dd54660a9d75e6e705874d9b482b226fc34dcf91287b6c0231fd72192824b9f; SignedEndpoints.lean ec046f04865f51355d4b560ec22880f80203f568e1bdf9a517a2a14d7faba2ac; Chain.lean 6e3fc2abae2badf60d797694c38679602f666315b4798573fd0b761d6225d055). Generators, bench files, C++ dump driver and instrumented-copy patch: results/proposal2/2026-10-26/A1/. LeanFlow commit 06084ef5c40057c66c476feba18ea3da8fb150ba. Pinned toolchain: Lean 4.34.1 and Mathlib [5] d13f23b7 (the openai/math stack).

```
python3 scripts/certified_numerics/lean_pinned.py compile formal_cert/Programmes/SandierSerfaty/Constants.lean --olean Programmes.SandierSerfaty.Constants
python3 results/proposal2/2026-10-26/A1/scripts/compile_seq.py [--keep-going] FILE[:Module.Name] ...   # file list in dependency order, one JSON log each
python3 scripts/certified_numerics/lean_pinned.py compile formal_cert/Programmes/SandierSerfaty/AxiomsAll.lean   # 615 axiom reports; non-whitelisted: none
python3 scripts/certified_numerics/lean_pinned.py compile formal_cert/Programmes/SandierSerfaty/ControlsReject.lean   # must exit 1 with the 7 mapped errors
.venv/bin/python -I -B -m pytest <LeanFlow worktree>/tests -q -p no:cacheprovider --rootdir <LeanFlow worktree>   # 94 passed
.venv/bin/python -I papers/p2_sandier_serfaty_lean/gen_inputs.py   # then ledger/build_ledger.py, ledger/run_gate.py, gen_inputs.py again, build_pdf.py
```

The logs, results and a copy of the two notes (CERTIFICATE_PORT.md, REDUCTION_LEMMA.md) are in papers/p2_sandier_serfaty_lean/data/; the ledger evidence blobs hash them. All numbers in this paper that are derived (shares, rates, projections, axiom counts, test counts) are generated by gen_inputs.py from those files into gen_numbers.tex. Compile times (process time, cores 4,5, load average 12–16): Constants 76 s, Bcons 287 s, the calcK files 1,281.8 s in total, the calcLam files 10,080.6 s; all genuine compile transcripts together (excluding the three must-fail files and the three axiom-audit files) sum to 12,267 s, i.e. 3.4 h.

13 Claim ledger

ledger/build_ledger.py files 35 claims: 16 at Tier A, 1 at L, 8 at C and 10 at X, each with a content-addressed evidence blob in ledger/evidence/: verbatim #print axioms lines and source hashes (A), a transcribed excerpt (L), the argument text and its file hashes (C), result files with claim_licence: false (X). The A rows rest on nothing; the C rows rest only on A and L rows and never on an X row; the X rows rest on A rows. The Elenchus gate (tools/ledger.py ledger.json --evidence-dir evidence, verbatim output in ledger/gate_output.txt) exits with code **1**: 0 blocking findings and 16 flags, all LEDGER_UNAUDITED_TIER_A (one per Tier A row: no person has certified that the statements mean what their glosses say). The flags are open and the audit fields are left null. The L row is a transcription of the P1 excerpt; citation_check.py was not run in this paper.

References

[1] E. Sandier, S. Serfaty, *From the Ginzburg–Landau model to vortex lattice problems*, Comm. Math. Phys. **313** (2012) 635–743; arXiv:1011.4617v2 [L 0001].

[2] OpenAI, *Triangular minimality for planar Coulomb renormalized energy* (preprint dated 2026-09-23), in the OpenAI Math release, github.com/openai/math, folder preprints/Triangular-minimality-for-planar-Coulomb-renormalized-energy-September-23-2026 (verification sources certificate.cpp, core.hpp, grids.hpp, sha256 prefixes 53d46bba, c7f76596, 4640e0a3). AI-generated; not peer reviewed.

[3] X. Callens, *The Sandier–Serfaty triangular-lattice conjecture: an independent reproduction of a claimed finite certificate, an adversarial audit of the claimed reduction, and a Lean statement (no proof of the conjecture is claimed)*, doi:10.5281/zenodo.23653827 (2026).

[4] X. Callens, *Can the planar Sandier–Serfaty proof architecture reach three-dimensional bcc jellium? A feasibility study with a negative outcome*, doi:10.5281/zenodo.23655002 (2026).

[5] The mathlib Community, *The Lean mathematical library*, CPP 2020; arXiv:1910.09336. *(Title, venue and arXiv number checked on the arXiv text.)*

The sibling feasibility study [4] assesses whether this proof architecture extends to three dimensions; it is cited for context only and nothing here depends on it.